Base facturation_db — service facturation

Hôtel & Restaurant — Dictionnaire des données V1 (MPD Merise)

Ce document explique **chaque table et chaque attribut** des 7 bases de données : ce qu'il contient, **pourquoi** il existe et **quelle logique** l'utilise. Il correspond au **MPD** (modèle physique) de Merise : les fichiers sources sont `services/<service>/prisma/schema.prisma` et les migrations SQL dans `services/<service>/prisma/migrations/`.

---

0. Les conventions (à lire en premier, elles valent pour toutes les tables)

Convention	Exemple	Pourquoi
Clé primaire `id` en UUID	`78072c82-869c-…`	Généré sans demander à la base, impossible à deviner (on ne peut pas essayer `id=1, 2, 3…` pour lire les données des autres), et unique **entre services** : un `reservationId` créé par hebergement peut voyager dans un événement sans collision.
Nom en camelCase dans le code, snake_case en base	`tarifNuit` ↔ `tarif_nuit` (`@map`)	Convention de TypeScript d'un côté, de PostgreSQL de l'autre. `@@map("chambre")` donne le nom de la table.
Montants en `Int` (entier), en Ariary	`85000`	L'Ariary n'a pas de centimes en pratique, et un entier n'a **jamais d'erreur d'arrondi** (contrairement à `0.1 + 0.2` en virgule flottante).
`createdAt` / `updatedAt`	`@default(now())` / `@updatedAt`	Traçabilité : quand la ligne a été créée, quand elle a changé pour la dernière fois. Prisma remplit `updatedAt` tout seul.
`Timestamptz` pour un instant, **`Date`** pour un jour	`checkInLe` vs `dateArrivee`	Un instant (« 14 h 32 à Tana ») a besoin du fuseau horaire ; une date de séjour (« le 15 octobre ») n'a pas d'heure. Mélanger les deux crée des décalages d'un jour.
`Json` pour les textes traduits (`TexteTraduit`)	`{"fr": "Chambre standard", "mg": "Efitra tsotra"}`	Une seule colonne pour toutes les langues. Ajouter l'anglais = ajouter une clé `en`, sans migration.
« sans FK » + « snapshot »	`clientId` (sans FK) + `clientNom` (snapshot)	**Règle de la SOA** : une base n'a pas de clé étrangère vers la base d'un autre service (ce sont des bases séparées). On garde l'**identifiant** (pour retrouver la donnée) et une **copie figée** des champs utiles (pour afficher sans appeler l'autre service, et pour garder la valeur **au moment de l'action** : si le client change de nom, sa facture garde l'ancien).
`version`	`version Int @default(1)`	**Verrou optimiste** (commande, reservation, tache_menage) : chaque modification fait `version + 1` et exige la version lue (`where: { id, version }`). Si deux personnes modifient en même temps, la seconde modifie 0 ligne et reçoit un conflit (409) au lieu d'écraser la première (« mise à jour perdue »). Pour `chambre`, l'incrément sert à **verrouiller la ligne** pendant le check-in (voir cette table).
Enum	`statut StatutReservation`	Liste fermée de valeurs, vérifiée par la base : une faute de frappe (`"CONFIMEE"`) est impossible.
Contraintes `CHECK` / `EXCLUDE` dans les migrations	`CHECK (quantite >= 0)`	Prisma ne sait pas les écrire dans `schema.prisma` : elles sont ajoutées **à la main** dans le SQL de migration. C'est la **dernière ligne de défense** : même un bug dans le code ne peut pas les violer.

Les 3 tables techniques (présentes dans plusieurs services)

`outbox_evenement` — patron *Transactional Outbox* (hebergement, facturation, menage, stock, restaurant, comptabilite)

Attribut	Type	Description et logique
`id`	uuid	Sert aussi d'**identifiant de l'événement** (les consommateurs s'en servent pour l'idempotence).
`type`	texte	Nom de l'événement : `reservation.creee`, `menage.termine`… (liste dans `packages/commun/src/evenements.ts`).
`payload`	json	Le contenu de l'événement, validé par un schéma Zod.
`createdAt`	instant	Ordre de publication.
`publieLe`	instant, **nul tant que non publié**	Le relais lit les lignes où `publieLe` est nul, les envoie dans Redis/BullMQ, puis remplit la date. Index `(publieLe, createdAt)` pour trouver vite les événements en attente.

Pourquoi : l'événement est écrit dans **la même transaction** que la donnée métier. Soit les deux sont enregistrés, soit aucun. Si Redis est en panne, l'événement attend ici et part plus tard : il n'est jamais perdu.

`evenement_traite` — consommateur idempotent

Attribut	Type	Description et logique
`evenementId`	uuid, **clé primaire**	L'id de l'événement reçu. Inséré **avant** de traiter, dans la même transaction que l'effet. Si le même événement arrive deux fois, l'insertion viole la clé primaire, la transaction est annulée : **aucun double effet** (jamais deux tâches de ménage pour le même séjour). Utilisé par la fonction `traiterUneFois` (`src/outbox.ts` de chaque service). Exception : stock s'appuie sur la clé unique métier `(referenceId, articleId)` de `mouvement`, une transaction par article.
`traiteLe`	instant	Quand il a été traité.

`compteur_<x>` (`compteur_reservation`, `compteur_facture`, `compteur_commande`)

Attribut	Type	Description et logique
`annee`	entier, **clé primaire**	Un compteur par année : la numérotation repart à 1 chaque 1er janvier.
`dernierNumero`	entier	Dernier numéro donné. Un `upsert` + `increment` **atomique** garantit que deux créations simultanées n'obtiennent jamais le même numéro → `RES-2026-000042`, `FAC-2026-000123`, `CMD-2026-000045`. Pour les factures, la numérotation **sans trou** est une obligation comptable.

3. Base `facturation_db` — service facturation (factures et paiements)

Table `facture`

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Identifiant.
`numero`	texte, **unique**	oui	`FAC-2026-000123`, séquentiel **sans trou** (`compteur_facture`, même transaction) : obligation comptable.

Attribut	Type	Obligatoire	Description et logique
`type`	enum `TypeFacture`	oui	`SEJOUR` (une réservation) ou `RESTAURANT` (une commande payée sur place).
`reservationId`	uuid, **unique**, sans FK	non	Pour une facture de séjour. **Unique** : une réservation n'a qu'**une** facture, même si l'événement `reservation.creee` arrive deux fois.
`commandeId`	uuid, **unique**, sans FK	non	Pour une facture restaurant.
`clientId`	uuid (auth), sans FK	non	Le compte du client : il ne voit et ne paie **que ses** factures.
`clientNom`, `clientEmail`, `clientTelephone`	texte (**snapshots**)	nom oui	Pour le reçu et pour pré-remplir le paiement mobile.
`langue`	enum	oui	Langue de la facture.
`statut`	enum `StatutFacture`	oui	`EMISE` → `PAYEE` (montant atteint) → `CLOTUREE` (séjour terminé, copie envoyée à la comptabilité) ; ou `ANNULEE` (réservation expirée). **Une facture émise ne se supprime jamais** (traçabilité comptable).
`montantTotal`	entier (Ar)	oui	Somme des lignes. Augmente si un repas est imputé à la chambre.
`montantPaye`	entier (Ar)	oui	Somme des paiements réussis. **CHECK `facture_pas_de_surpaiement` : `montant_paye <= montant_total`** — la base refuse d'encaisser plus que dû (ex. espèces à la réception + Papi en même temps).
`emiseLe`, `clotureeLe`	instant	| Dates comptables.	

Reste à payer = `montantTotal - montantPaye` (calculé, jamais stocké : une valeur calculée stockée finit toujours par être fausse).

Table `ligne_facture`

Attribut	Type	Description et logique
`id`	uuid	
`factureId`	uuid, FK → facture (cascade)	
`libelle`	texte	« Suite Prestige – 3 nuits », « Restaurant – CMD-2026-000045 ».
`quantite`, `prixUnitaire`, `montant`	entiers, CHECK > 0 / ≥ 0	`montant = quantite × prixUnitaire`.
`origine`	enum `Origine`	**À qui appartient l'argent** : `HEBERGEMENT`, `RESTAURANT`, `LIVRAISON` (V3). Un repas imputé à la chambre est **payé à l'hôtel** mais **appartient au restaurant** → la comptabilité crée un reversement.
`sourceId`	uuid, sans FK	L'id de la réservation ou de la commande facturée.

Attribut	Type	Description et logique
Unique	`(origine, sourceId)`	Une commande n'est facturée **qu'une fois**, même si `consommation.imputee` est rejoué.

Table `paiement`

Attribut	Type	Obligatoire	Description et logique
`id`	uuid	oui	Envoyé à Papi comme `reference` ; Papi le renvoie dans `merchantPaymentReference` → vérification de cohérence.
`factureId`	uuid, FK → facture	oui	La facture payée (plusieurs paiements possibles : un acompte Papi + le reste en espèces).
`montant`	entier, CHECK > 0	oui	**Le reste à payer calculé par le serveur** au moment de la demande. Le montant notifié par Papi doit être **exactement** celui-ci, sinon `ANOMALIE`.
`statut`	enum `StatutPaiement`	oui	`EN_ATTENTE` (lien créé) → `REUSSI` / `ECHOUE` ; `ANOMALIE` = argent reçu mais incohérent (mauvais montant, facture déjà soldée, lien remplacé) → **remboursement manuel**.
`methode`	enum `MethodePaiement`	non	`ESPECES`, `MVOLA`, `ORANGE_MONEY`, `AIRTEL_MONEY`, `CARTE`. Inconnue tant que Papi n'a pas dit comment le client a payé.
`referenceExterne`	texte, **unique**	non	`paymentReference` de Papi : une même transaction Papi ne peut être enregistrée **qu'une fois**.
`encaissePar`	uuid (auth), sans FK	non	Pour les espèces : qui a encaissé (contrôle de caisse).
`jetonHash`	texte	non	**Empreinte SHA-256** du jeton secret placé dans l'URL de notification. **1re vérification** d'une notification : elle doit présenter le bon jeton. On ne garde que l'empreinte : même avec une copie de la base, impossible de fabriquer une notification valide.
`notificationTokenHash`	texte	non	Empreinte du `notificationToken` que Papi a renvoyé à la création du lien. **2e vérification**, faite par le worker.
`urlPaiement`	texte	non	L'adresse de la page Papi, pour la renvoyer en cas de double clic (règle corrigée par **B2**).
`erreur`	texte	non	Pourquoi le paiement a échoué (« Solde insuffisant », « Remplacé par un nouveau lien »…).

Index unique partiel `paiement_un_en_attente_par_facture` (SQL écrit à la main) :

```
CREATE UNIQUE INDEX … ON paiement (facture_id) WHERE statut = 'EN_ATTENTE';
```

→ **au plus un** paiement en attente par facture : trois clics simultanés ne créent pas trois liens Papi.

---

9. Ce qui changera avec le SaaS (Phase 2)

- **Toutes** les tables métier recevront `hotelId` (sauf les tables propres au client global).
- Les uniques deviendront **par hôtel** : `(hotelId, numero)` pour les chambres, `(hotelId, code)` pour les catégories et les articles, compteurs `(hotelId, annee)`.

- `utilisateur` deviendra **global** ; une nouvelle table `membre (utilisateurId, hotelId, role)` portera les rôles du personnel.
- `outbox_evenement` et les événements porteront le `hotelId`.
- Les politiques **RLS** de PostgreSQL filtreront par `hotelId` (personnel) ou par `clientId` (client).